

GESTIÓN DE CONFIGURACIÓN DE SOFTWARE

SISTEMA DE GESTIÓN DE TURNOS Y TRIAJE MÉDICO AUTOMATIZADO

CARAL DANIELA MARTÍNEZ PANQUEVA

CC 1127045653

INGENIERÍA DE SISTEMAS

FACULTAD DE INGENIERÍAS Y ARQUITECTURA

UNIVERSIDAD DE PAMPLONA

JUNIO, 2026

1. ESTRATEGIA DE RAMAS: REGLAS DEFINIDAS PARA GIT (MODELO GITFLOW)

Para el desarrollo, mantenimiento y despliegue del sistema MedTriage, se adopta estrictamente el modelo de ramificación Gitflow. Dado que la aplicación gestiona flujos críticos de salud en el servicio de urgencias de Villa del Rosario, esta estrategia mitiga el riesgo de introducir errores en el entorno de producción y permite el trabajo simultáneo en los módulos de Admisión, Enfermería y Médicos.

A. Definición y Ciclo de Vida de las Ramas

- **Rama main (Producción):**

- ✓ **Propósito:** Aloja el código fuente definitivo, estable y 100% operativo que se encuentra desplegado en el servidor local.
- ✓ **Reglas de Gobierno:** Está prohibido realizar *commits* directos. Solo recibe código proveniente de develop mediante *Pull Requests* (PR) aprobados tras superar las pruebas de regresión, o de ramas hotfix/*. Cada estado estable en esta rama se etiqueta obligatoriamente con una versión semántica (Ej: v1.0.0).

- **Rama develop (Integración/Desarrollo):**

- ✓ **Propósito:** Es la rama centralizada de consolidación. Contiene las últimas características desarrolladas que están listas para la fase de pruebas de QA.
- ✓ **Reglas de Gobierno:** Recibe fusiones de las ramas feature/*. Todo código integrado aquí debe compilar limpiamente sin errores y mantener la cobertura de pruebas requerida.

- **Ramas feature/* (Funcionalidades de Casos de Uso):**

- ✓ **Propósito:** Ramas temporales destinadas exclusivamente al desarrollo de una característica o requisito específico. Nacen a partir de develop y mueren al integrarse de nuevo en ella.
- ✓ **Nomenclatura Estricta:** Se estructuran usando el identificador del Caso de Uso (CU) documentado en la matriz de trazabilidad:
 - feature/CU-001-autenticacion: Implementación de login y tokens JWT.

- feature/CU-002-registro-pacientes: Formulario de captura demográfica de admisión.
 - feature/CU-003-signos-vitales: Formulario de constantes biológicas para enfermería.
 - feature/CU-006-algoritmo-manchester: Lógica central del cálculo de niveles de triaje.
- **Ramas hotfix/* (Parches Críticos en Caliente):**
 - ✓ **Propósito:** Ramas de emergencia que nacen directamente de main cuando se detecta un fallo catastrófico en la operación real de urgencias (Ej: el desplome del WebSocket en la pantalla pública de turnos o un error HTTP 500 al guardar la evolución médica).
 - ✓ **Reglas de Gobierno:** Una vez solucionado el fallo, la rama se fusiona de manera inmediata tanto en main (para actualizar producción) como en develop (para evitar que el error vuelva a aparecer en futuros despliegues).

B. Flujo de Trabajo y Políticas de Integración

- 1) **Políticas de Code Review:** Para fusionar cualquier rama feature/* hacia develop, es obligatorio abrir un *Pull Request*. Este debe ser revisado por al menos un compañero de equipo para garantizar la legibilidad del código y el cumplimiento del patrón arquitectónico MVC.
- 2) **Criterio de Aceptación Técnico:** No se permite la integración de código que rompa las pruebas unitarias existentes o que degrade la cobertura automatizada por debajo del límite establecido en el plan de calidad.

2. INVENTARIO DE ARTEFACTOS (LISTA DE PAQUETES Y DEPENDENCIAS)

A continuación, se detalla el inventario técnico de paquetes, dependencias y bibliotecas de terceros requeridas para compilar y ejecutar el ecosistema de MedTriage. El proyecto se divide bajo un enfoque desacoplado, gestionando dependencias independientes a través de Maven (`pom.xml`) en el backend y NPM (`package.json`) en el frontend.

A. Artefactos del Servidor Backend (Ecosistema Java / Spring Boot 3.x)

Grupo / Artefacto ID	Versión	Tipo / Propósito	Justificación Técnica en MedTriage
org.springframework.boot:spring-boot-starter-web	3.2.x	Núcleo de Red / API	Proporciona el servidor embebido Tomcat y las herramientas para exponer los controladores RESTful que comunican el frontend con la lógica del negocio mediante JSON.
org.springframework.boot:spring-boot-starter-security	3.2.x	Seguridad y Ciberseguridad	Gestiona el ciclo de autenticación y autorización. Bloquea los endpoints sensibles e intercepta las peticiones controlando el acceso estricto por roles (Admisión, Enfermería, Médico).
io.jsonwebtoken:jjwt-api	0.11.5	Criptografía / Sesiones	Implementa la generación y validación de tokens firmados digitalmente (JWT), garantizando un intercambio de información sin estado (<i>stateless</i>) y seguro entre cliente y servidor.
org.springframework.boot:spring-boot-starter-data-jpa	3.2.x	Persistencia de Datos	Framework ORM (Hibernate) encargado de mapear las tablas relacionales de la base de datos (Historias clínicas, turnos, signos vitales) a objetos Java, optimizando las consultas.
org.postgresql:postgresql	42.6.x	Driver de Conexión	Conector JDBC oficial necesario para que la aplicación Spring Boot establezca comunicación síncrona y persistente con el motor de base de datos PostgreSQL 15+.
org.jacoco:org.jacoco.core	0.8.11	Aseguramiento de Calidad	Herramienta de cobertura de código embebida en la fase de testeo para auditar analíticamente que las pruebas de software verifiquen correctamente los algoritmos de clasificación.

B. Artefactos del Cliente Frontend (Ecosistema TypeScript / Angular 17+ / Node)

Nombre del Paquete (NPM)	Versión	Tipo / Propósito	Justificación Técnica en MedTriage
@angular/core & @angular/common	^17.0.0	Arquitectura Base UI	Ecosistema central del framework para construir la interfaz de usuario basada en componentes bajo la estructura SPA (Single Page Application).
@angular/router	^17.0.0	Enrutamiento y Guards	Administra la navegación web interna. Incluye <i>Guards</i> de seguridad en el navegador para denegar el acceso visual a interfaces médicas si el usuario logueado posee rol de Admisión.
rxjs	^7.8.x	Programación Reactiva	Maneja flujos de datos asíncronos mediante observables. Es fundamental para actualizar la cola de espera de pacientes en tiempo real cuando enfermería clasifica un nuevo caso.
@angular/common/http	^17.0.0	Cliente de Red	Módulo especializado para consumir los servicios web de la API REST del backend, inyectando de forma automatizada los headers con el token JWT de la sesión.